

Fraud Ring Detection

Capstone Project Report

27 September 2026

Team (Group 8): Nada Aljaafari, Lujain Alqarni, Alia AlGhamdi, Remas Almutairi

Summary

Our final model is XGBoost with five historical relationship features. On the most recent 20% of transactions, it reaches an Average Precision (AP) of 0.521 and a ROC-AUC of 0.909. A random model would score an AP of 0.034, so ours ranks fraud about 15 times better than chance.

At the default 0.50 threshold, the model catches 74% of fraud (3,014 of 4,064 cases), but only about 1 in 5 alerts is real fraud. Moving the threshold lets a fraud team trade missed fraud against false alerts.

The network analysis found real coordinated patterns. The largest group was 35 fraudulent transactions on one card and one address over about three days. Turning these relationships into features gave XGBoost only a small improvement, a neural network did not benefit, and the graph neural networks we tried (GraphSAGE and RGCN) did not beat XGBoost. The main lesson is that a well-tuned tabular model already captures most of the signal. Relational features help a little, and the added complexity has to be justified.

We also built **MAKĀMIN** | مكامن, a Streamlit dashboard where an investigator picks a transaction and sees its risk score, the main reasons behind it, and the related transactions from the previous 24 hours.

1. Problem and objectives

Fraud is not always one suspicious transaction. It is often a group of transactions that share a card, address, email or device, and each can look normal on its own. Our proposal asked whether relationships between transactions improve fraud detection compared with transaction-level machine learning.

Our objectives were to:

1. Explore how fraudulent and legitimate transactions differ.
2. Build a baseline XGBoost model.
3. Use network analysis to find coordinated fraud.
4. Add time, so that only close-in-time relationships count.
5. Engineer relationship features without using the fraud label.
6. Test whether those features improve performance.
7. Explain the final model with SHAP.

Change from the proposal. The proposal planned two graph neural networks, GraphSAGE and RGCN, and a Streamlit dashboard. We tried both GNNs, but neither performed better than XGBoost, so they are not part of the final notebook. We kept a feed-forward neural network (ANN) as the deep learning comparison, because it uses the same tabular features and chronological split as XGBoost. The dashboard was built as planned (Section 9).

2. Data

We used the [IEEE-CIS Fraud Detection dataset](#) (IEEE-CIS and Vesta Corporation, 2019). It holds 590,540 labelled e-commerce transactions.

- **Transaction table:** 590,540 rows, 394 columns, including amount, product code, card details, address, email domain, and many anonymised count (C), time-delta (D), match (M) and engineered (V) features.
- **Identity table:** 144,233 rows, 41 columns, including device type and device information.

We merged the two tables on TransactionID using a left join, so every transaction was kept even without identity data. The result was 590,540 rows and 434 columns, with no duplicate transaction IDs. The target is isFraud.

3. Exploratory analysis

Fraud is rare: 20,663 transactions (3.5%) are fraudulent. A model that always predicts "not fraud" would be 96.5% accurate and useless, so we judge models by Average Precision and ROC-AUC rather than accuracy.

Missing data is heavy. 414 of 434 columns have missing values, and nine identity columns are more than 99% empty. Much of this is structural: 76% of transactions have no identity record at all.

Amount alone separates the classes poorly. Fraudulent transactions have a median of \$75 against \$68.50 for legitimate ones, with similar spread.

Fraud rates vary much more by transaction type, card type and device:

Characteristic	Group	Transactions	Fraud rate
Product code	C	68,519	11.7%
Product code	W (most common)	439,670	2.0%
Card network	Discover	6,651	7.7%
Card network	Visa	384,767	3.5%
Card type	Credit	148,986	6.7%
Card type	Debit	439,938	2.4%
Device type	Mobile	55,645	10.2%
Device type	Desktop	85,165	6.5%
Device type	Missing	449,730	2.1%

Product C and mobile devices carry fraud rates about three to five times the overall 3.5%. Missing device information is itself informative: those transactions have the lowest fraud rate. These are associations, not causes.

4. Preparation and validation design

We split the data by time, not at random. Transactions were sorted by TransactionDT: the earliest 80% (472,432) became the training set and the latest 20% (118,108) the validation set. The periods do not overlap. This mirrors real use, where a model trained on the past must score future transactions. A random split would let the model learn from later fraud when predicting earlier fraud. Fraud rates were similar in both periods (3.51% and 3.44%).

Every preprocessing decision was learned from the training period only:

- **Sparse columns:** 9 identity columns with more than 95% missing values in training were dropped, leaving 423 features.
- **Categorical features (29):** missing values became "Unknown", then each category was mapped to a number using training categories only. Categories first seen in validation were coded as -1.
- **Numerical features (394):** kept as they are for XGBoost, which handles missing values natively. For the ANN, missing values were filled with training medians and features were standardised with a scaler fitted on training data.
- **Excluded:** TransactionID (an identifier) and the target isFraud.
- **Class imbalance:** fraud was up-weighted by 27.5 ($455,833 \text{ legitimate} \div 16,599 \text{ fraud in training}$) for both XGBoost and the ANN. We chose weighting over over- or under-sampling so the model still trains on the real data distribution.

5. Fraud network analysis

Fraudulent transactions do form groups. We built a network in which each node is a fraudulent transaction, then tested which shared attributes make good links.

Single attributes linked too broadly. For example, 86% of fraud shares an email domain with other fraud, but a domain like gmail.com connects thousands of unrelated people. Combinations were more specific:

Linking rule	Fraud transactions linked
Card + email domain	80.0%
Card + address	55.4%
Card + address + email domain	40.7%
Card + device	32.9%
Card + address + device	10.1%

We chose card + address as the base link and added time: two fraudulent transactions are connected only if they share both and happen within 24 hours. Edge weight rises from 1 to 3 when they also share the email domain, the device, or both.

The resulting network has 12,922 transactions, 12,226 links and 7,519 separate groups. 5,081 transactions stood alone. Most links are strong: 60% have weight 2 and 11% weight 3.

The largest group contains 35 fraudulent transactions totalling \$6,305. All use one card, one address and one email domain, across three devices over 73 hours. The picture shows two dense bursts of activity joined by a few transactions, which is consistent with one actor returning for a second round.

Largest Time-Aware Fraud Transaction Component 35 Transactions

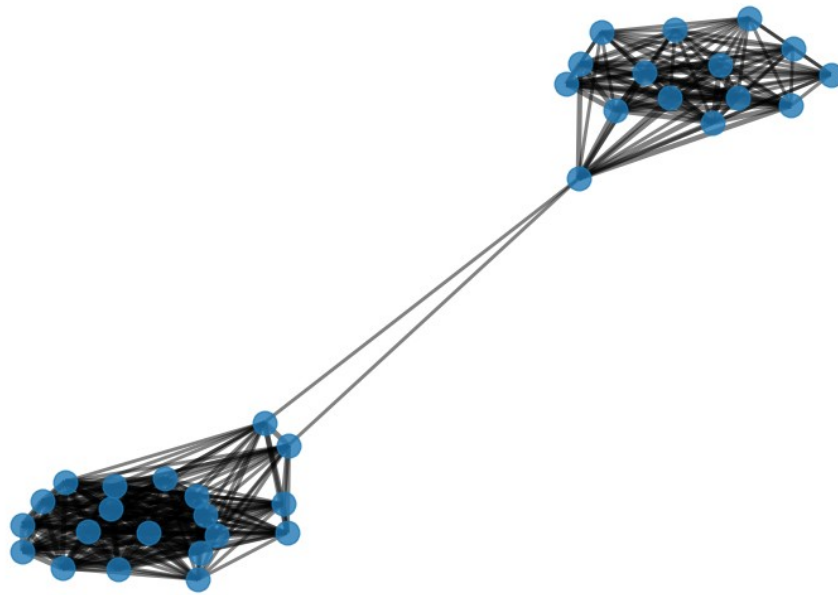


Figure 1. Largest time-aware fraud component: 35 transactions in two dense clusters

This network uses the fraud labels, so it was used only to understand the data. Building model features from it would leak the answer into the model. Section 6 describes the label-free version we used instead.

6. Feature engineering

We turned the network idea into five features that never use the fraud label. For each transaction, each feature counts how many earlier transactions in the previous 24 hours shared an identifier. Only past transactions are counted, so the model sees only information that would exist at the moment of scoring.

Feature	Counts earlier transactions in 24 h with the same...	Transactions with at least one match
card1_prev_count_24h	card	78.4%
addr1_prev_count_24h	address	88.7%
email_prev_count_24h	email domain	83.7%
device_prev_count_24h	device	17.8%
card_addr_prev_count_24h	card and address	49.5%

The counts were computed in time order over the full dataset, then split at the same point as the model data. We checked that transaction IDs lined up exactly in both periods. This moves the proposal's "neighbourhood fraud density" idea to a safe form: it measures how busy an identifier is, not how much fraud surrounds it, because counting known fraud would leak the label.

A separate check in the notebook (step 9.1.1) confirmed that the data is in strict time order, that the training period ends before validation begins, and that the fraud label is not used anywhere in these features.

7. Models and comparison

XGBoost beat the neural network on both metrics, with or without the relationship features. We trained four models on the same chronological split. Each pair differs only in the five historical features, so any change comes from them.

- **XGBoost:** 300 trees, depth 6, learning rate 0.05, 80% row and column sampling, fraud weight 27.5.
- **ANN:** three dense layers (256, 128, 64 units) with batch normalisation and dropout (0.2–0.3), a sigmoid output, Adam optimiser, batch size 2,048, and the same fraud weight. Training used early stopping on validation PR-AUC and halved the learning rate when it stalled.

Model	Features	Average Precision	ROC-AUC
XGBoost + historical features	428	0.521	0.909
XGBoost baseline	423	0.519	0.908
ANN baseline	423	0.451	0.861
ANN + historical features	428	0.440	0.873
Random guess	–	0.034	0.500

We ranked models by Average Precision first. It measures how well the model ranks the rare fraud cases at the top, which is what a fraud team works from. ROC-AUC can look strong even when many alerts are false, because legitimate transactions vastly outnumber fraud.

What the comparison shows:

- **XGBoost is about 0.07 AP ahead of the ANN.** Tree models suit tabular data with mixed types, many missing values and anonymised features. The ANN needed imputation and scaling, and its validation PR-AUC swung between epochs, which suggests unstable training.
- **The historical features helped XGBoost slightly** (+0.0017 AP, +0.0015 ROC-AUC).
- **For the ANN, the effect was mixed:** ROC-AUC rose by 0.011 but AP fell by 0.011.

We also tried the two graph neural networks from the proposal, GraphSAGE and RGCN. Neither beat XGBoost, so they were not carried into the final notebook.

8. Final model: XGBoost with historical features

We selected XGBoost with the five historical features. It had the highest Average Precision and ROC-AUC of all four models. It is also fast to train, handles missing data natively, and can be explained with SHAP. The gain over plain XGBoost is small, so plain XGBoost remains a reasonable simpler alternative.

Choosing a decision threshold

The model outputs a fraud probability, and a threshold turns it into an alert. At 0.50 it flags 14,122 transactions, catches 3,014 of 4,064 frauds (74% recall) and is right on 21% of its alerts.

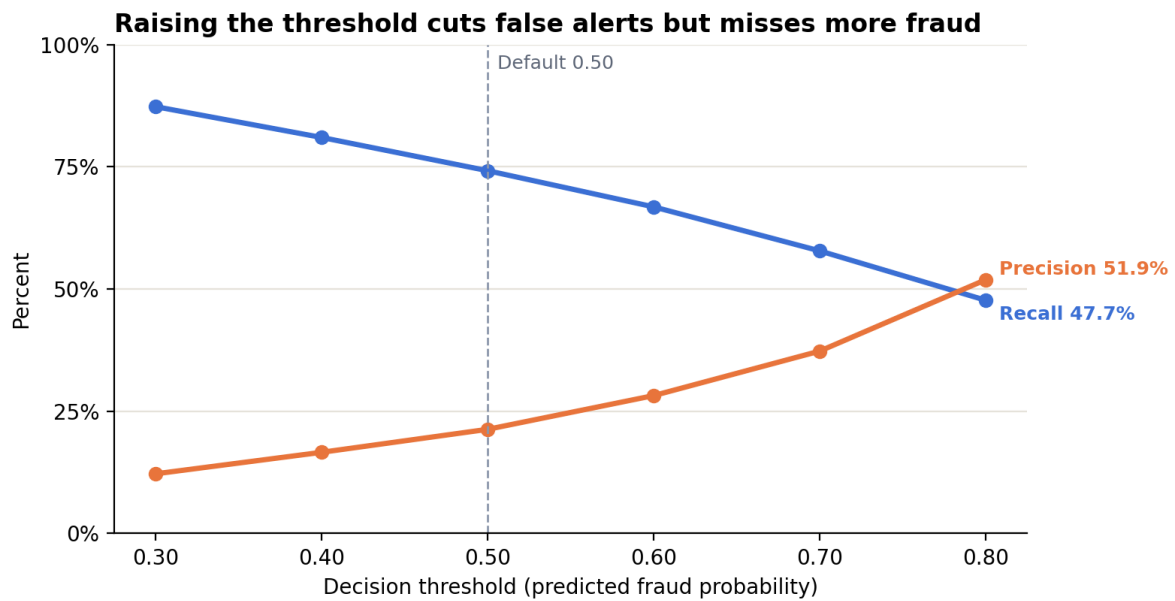


Figure 2. Precision and recall at each decision threshold (118,108 validation transactions)

Threshold	Fraud caught	Fraud missed	False alerts
0.30	3,546	518	25,511
0.50	3,014	1,050	11,108
0.70	2,350	1,714	3,948
0.80	1,938	2,126	1,796

The right threshold is a business decision. A bank that must stop most fraud would pick around 0.30–0.50 and staff a review team for the alerts. A team with little review capacity would pick 0.70–0.80, which cuts false alerts by 65–85% but misses about half the fraud. Overall accuracy at 0.50 is 89.7%, but that number mostly reflects the legitimate majority. We keep 0.50 as a reference threshold for the dashboard and the demo, not as a tuned production setting.

What drives the predictions (SHAP)

We used SHAP on 5,000 validation transactions to see which features push predictions up or down.

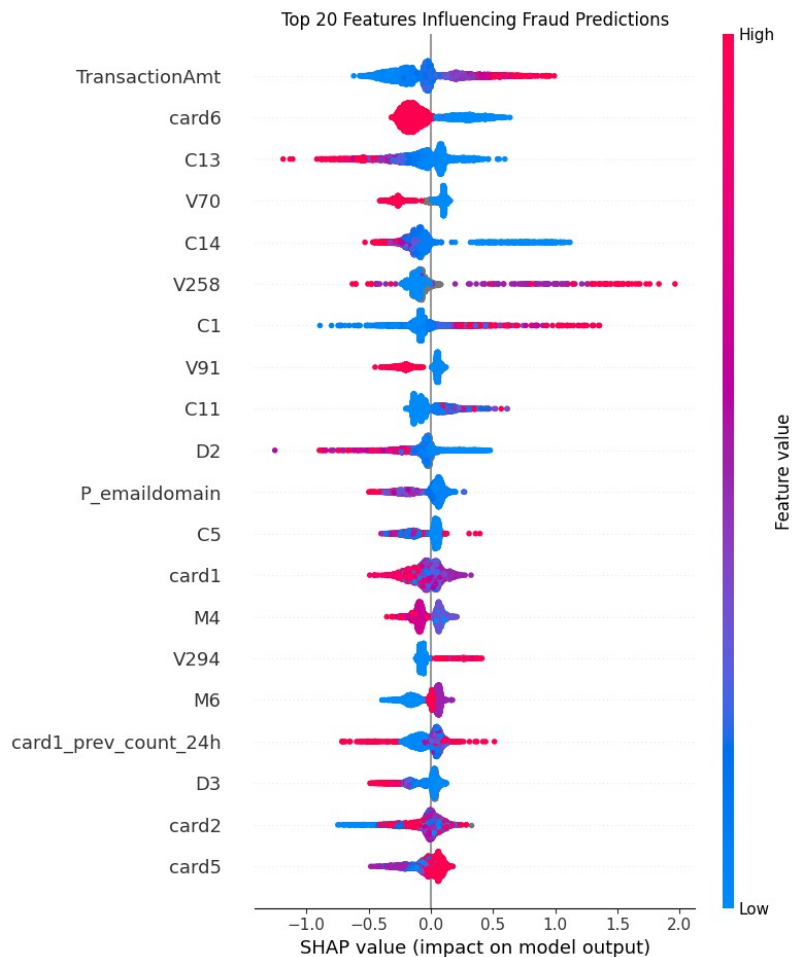


Figure 3. SHAP summary: top 20 features influencing fraud predictions

- **Transaction amount is the strongest single driver.** Higher amounts tend to raise the fraud score.
- **Card type (card6) is second,** matching the EDA finding that credit cards have almost three times the fraud rate of debit cards.
- **Anonymised count features** (C13, C14, C1, C11) and Vesta's engineered V features fill most of the top 10. They carry strong signal, but their exact meaning is hidden, which limits how far we can explain them.
- **card1_prev_count_24h ranks 17th of 428 features,** the highest of our five historical features. The others rank 36th (address), 56th (email), 61st (device) and 82nd (card + address).

The historical features act in a non-linear way. More recent activity on a card does not always mean more risk; its effect depends on the other features. So the relationship features are used by the model, but as a supporting signal rather than a main driver.

9. MAKĀMIN investigation dashboard

We turned the final model into an interactive investigation tool, **MAKĀMIN | مكامن — Uncovering Hidden Fraud Patterns**. An investigator selects a transaction and moves from prediction, to explanation, to related transactions:

1. **Risk assessment:** the final XGBoost fraud score and its decision at the 0.50 reference threshold, shown next to the known label for comparison.
2. **Explanation:** the 10 features with the largest SHAP contributions for that transaction, and whether each one pushes the score toward or away from fraud.

3. **Relational investigation:** earlier transactions from the previous 24 hours that share its card, address, email domain or device, drawn as an interactive network. Future transactions are never included, and fraud labels are added only after the links are found.

The dashboard covers the 5,000 validation transactions that have SHAP explanations. It is built with Streamlit, Plotly and PyVis. For large neighbourhoods, the network shows up to 150 related transactions, while the summary numbers use all of them. For the demo, it runs from Kaggle through a temporary public link.

MAKĀMIN is a demonstration and investigation aid, not a production system. A link between two transactions is context for an investigator, not proof of coordinated fraud.

10. Reliability checks

The results come from a leakage-aware setup, but the gap between our two XGBoost models is too small to call a clear win.

Check	What we did	Result
Performance on new data	Trained on the earliest 80%, tested on the latest 20%	AP 0.521 on unseen future transactions
Data leakage	Time-based split, training-only preprocessing, past-only features, labelled fraud network kept out of the model, plus a notebook check of time order	No known leakage into XGBoost
Metric choice	AP and ROC-AUC, plus precision, recall and F1 at set thresholds	Suited to 3.5% fraud; accuracy not used to rank models
Overfitting (ANN)	Dropout, batch normalisation, early stopping	Validation PR-AUC jumped between epochs, so training was unstable
Overfitting (XGBoost)	Row and column sampling, depth limit of 6	Training scores not recorded, so the train-validation gap is unknown

Three points weaken the conclusions and should be stated openly:

1. **The validation set was used to steer the ANN.** Early stopping and learning-rate cuts watched validation PR-AUC, so the ANN scores are slightly optimistic. The threshold comparison also used the same data. A separate final test period would give cleaner numbers.
2. **One split, one run.** The +0.0017 AP gain from historical features may be within random variation. Repeating with time-based cross-validation or several seeds would show whether it holds.
3. **Features computed across both periods.** The 24-hour counts were built over the full timeline, so early validation transactions can count late training ones. This uses no labels and matches real scoring, so it is not leakage, but it is worth mentioning.

How performance developed. The baseline XGBoost reached AP 0.519. Adding the historical features raised it to 0.521. The ANN reached 0.451 and dropped to 0.440 with the new features, so XGBoost was kept as the final model.

11. Limitations, future work and conclusion

Limitations

- **GNN results are not documented.** We tried GraphSAGE and RGCN and neither beat XGBoost, but those experiments are not in the final notebook, so their scores are not reported here.
- **Anonymised features.** Many top SHAP features (C, D, V columns) have hidden meanings, which limits business interpretation.
- **Identifiers are imperfect.** card1 and addr1 are not true customer IDs, so some links join unrelated people and others miss real repeat actors.
- **Little tuning.** Both models used one set of hyperparameters; the ANN trained on CPU only.
- **Labels arrive late in practice.** Real fraud labels come days or weeks after a transaction, which our set-up does not simulate.

Future work

1. Revisit graph neural networks with a richer entity graph and more tuning, and report them with the same split and metrics as XGBoost.
2. Use time-based cross-validation and a held-out final test period.
3. Tune XGBoost hyperparameters and report training scores to check the overfitting gap.
4. Build stronger customer identifiers (for example card + address + account start date) and group-level features.
5. Pick the threshold by cost: the average loss from missed fraud against the cost of reviewing an alert.
6. Host MAKĀMIN permanently instead of through a temporary Kaggle link, and extend it beyond the 5,000-transaction demo sample.

Conclusion

Our hypothesis was that relationship information would improve fraud detection. The results partly support it. The network analysis showed real coordinated fraud, such as 35 transactions on one card and address over three days. Label-free historical features were used by the model, with the card count ranking 17th of 428 in SHAP. However, they improved XGBoost only slightly, did not help the ANN, and the graph neural networks we tried did not beat XGBoost either.

The final model, XGBoost with historical features, detects 74% of fraud at the default threshold with an AP of 0.521, about 15 times better than chance. The most complex model was not the best one: the tree-based model beat the neural network on this tabular data. Relationship features are a sound, low-risk addition, but on this data a well-tuned tabular model is hard to beat. MAKĀMIN shows how the model could be used in practice, by putting the score, its reasons and the related transactions in front of an investigator in one place.

References

- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD Conference*, 785–794.
- IEEE-CIS & Vesta Corporation. (2019). [IEEE-CIS Fraud Detection dataset](#). Kaggle.
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* 30, 4765–4774.

- Cheng, D., Zou, Y., Xiang, S., & Jiang, C. (2024). Graph neural networks for financial fraud detection: A review. *arXiv:2411.05815*.